

CSCI 12532

Computer Architecture and Design

Lecture 09

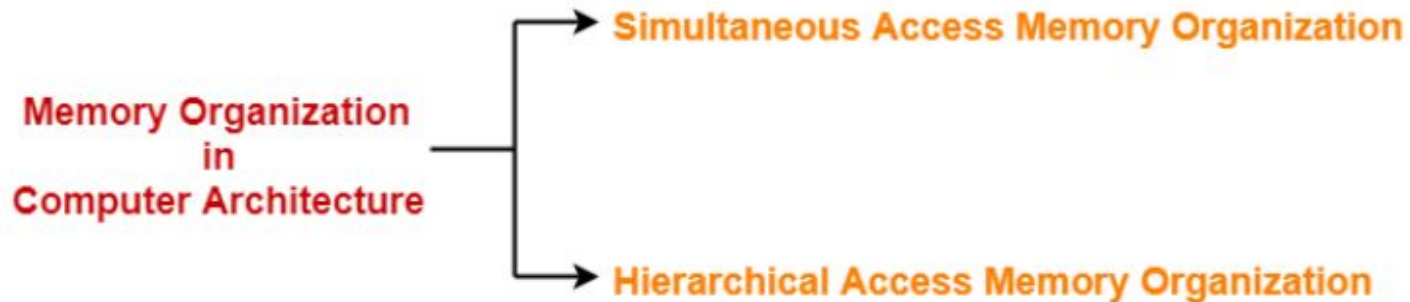
Ms. Meesha Mervyn
Dept. of Computer Systems Engineering
Faculty of Computing and Technology
University of Kelaniya
meesham@kln.ac.lk

Contents

1. Memory Organization
2. Memory Management
3. Memory Hierarchy
4. Cache Memory
5. Virtual Memory

1. MEMORY ORGANIZATION

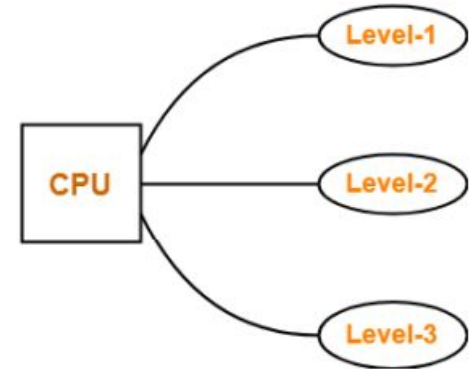
Memory organization refers to the way that the computer's memory is arranged and managed. The memory of system can be thought of as a large number of addressable storage locations. Each location can store a fixed amount of data, typically measured in bits or bytes. The way these locations are organized and accessed can have a notable impact on the overall production and functionality of the system.



Types of Memory Organization

1. Simultaneous Access Memory Organization

- All the levels of memory are directly connected to the CPU. Whenever CPU requires any word, it starts searching for it in all the levels simultaneously.
- Systems use parallel access techniques, such as
 - Memory Interleaving (Banking): This technique breaks memory into multiple independent modules (banks) that can be accessed simultaneously.
 - Memory Coalescing: Specifically used in GPUs to combine memory requests from multiple threads into a single transaction.
 - Conflict-Free Address Generation: For specialized processors like FFT (Fast Fourier Transform), this technique utilizes address generation lookup tables to ensure that multiple memory banks are accessed concurrently without conflicts.



2. Hierarchical Access Memory Organization

In this memory organization, memory levels are organized as,

- Level-1 (L1 Cache): Fastest and smallest, directly integrated into the CPU.
- Level-2 (L2 Cache): Larger than L1 but slower, still on the CPU chip.
- Level-3 (L3 Cache): Shared among CPU cores, larger but slower than L2.
- Main Memory (RAM): Much larger but slower than cache, connected to the CPU via the memory bus.
- Secondary Storage (SSD/HDD): Non-volatile, large, and slow, used for long-term data storage.

Ex;

Cache-to-Register Flow: When the CPU requires data, it first checks the Cache (L1). If the data is not present (cache miss), it is fetched from the Main Memory (RAM), transferred to the Cache, and then to the register.

Virtual Memory (HDD/SSD to RAM): When RAM is full, the operating system moves infrequently used data to a swap file on the HDD or SSD. When that data is requested again, it is moved back into RAM.



1. Memory Management

Memory management is an important issue while designing a computer system . Efficient memory management is vital in a multiprogramming system . If only a few process are in memory, then for much of the time all of the process will be waiting for I/O and the processor will idle. Thus memory needs to be allocated efficiently to pack as many processes into main memory.

Memory management is the process of allocating, tracking, and deallocating memory resources in a computer system to ensure efficient and secure operation.

We use memory to store the information, which includes both program and data. Due to several reasons, we have different kind of memories. We use different kind of memory at different level. The memory of computer is broadly categories into two categories: Internal and external.

Goals of Memory Management

1. Optimal Memory Utilization
 - Efficiently allocate memory to processes and reclaim it when no longer needed.
 - Minimize fragmentation (both internal and external) to maximize usable memory.
 - Use dynamic memory allocation techniques

2. Memory Protection
 - Ensure process isolation to prevent one process from accessing or corrupting another process's memory.
 - Implement hardware-enforced mechanisms like Memory Management Units (MMUs) and page tables.
 - Use address space randomization (ASLR) to prevent security exploits.

3. Memory Sharing

- Enable efficient sharing of memory resources among processes (e.g., shared libraries, inter-process communication).
- Use copy-on-write (COW) techniques to optimize shared memory usage.

4. Memory Organization

- Divide memory into segments and track their allocation status.
- Use paging and segmentation to manage memory in fixed-size blocks or variable-sized segments.

Techniques in Memory Management

1. Single contiguous

Single contiguous memory management schemes are one of the oldest techniques. In this technique, the main memory has two partitions: one for the OS and the other for different user processes. However, due to its simplicity, single contiguous memory management leads to memory waste. While it played an important role in developing more advanced systems, this technique isn't practical for modern computers.

2. Fixed partitioning: Functions by breaking the memory into partitions of equal size. Each program, regardless of its size, utilizes an entire partition. Fixed partitioning: Functions by breaking the memory into partitions of equal size. Each program, regardless of its size, utilizes an entire partition.

3. Dynamic partitioning: Functions by using partitions of various sizes, and each process is only given the amount of memory it needs to operate.

Non-contiguous

4. Paging

In the paging memory management technique, blocks are organized into frames, which are specific-sized blocks of physical memory. The frames do not need to be contiguous, reducing the amount of external fragmentation that occurs in the main memory.

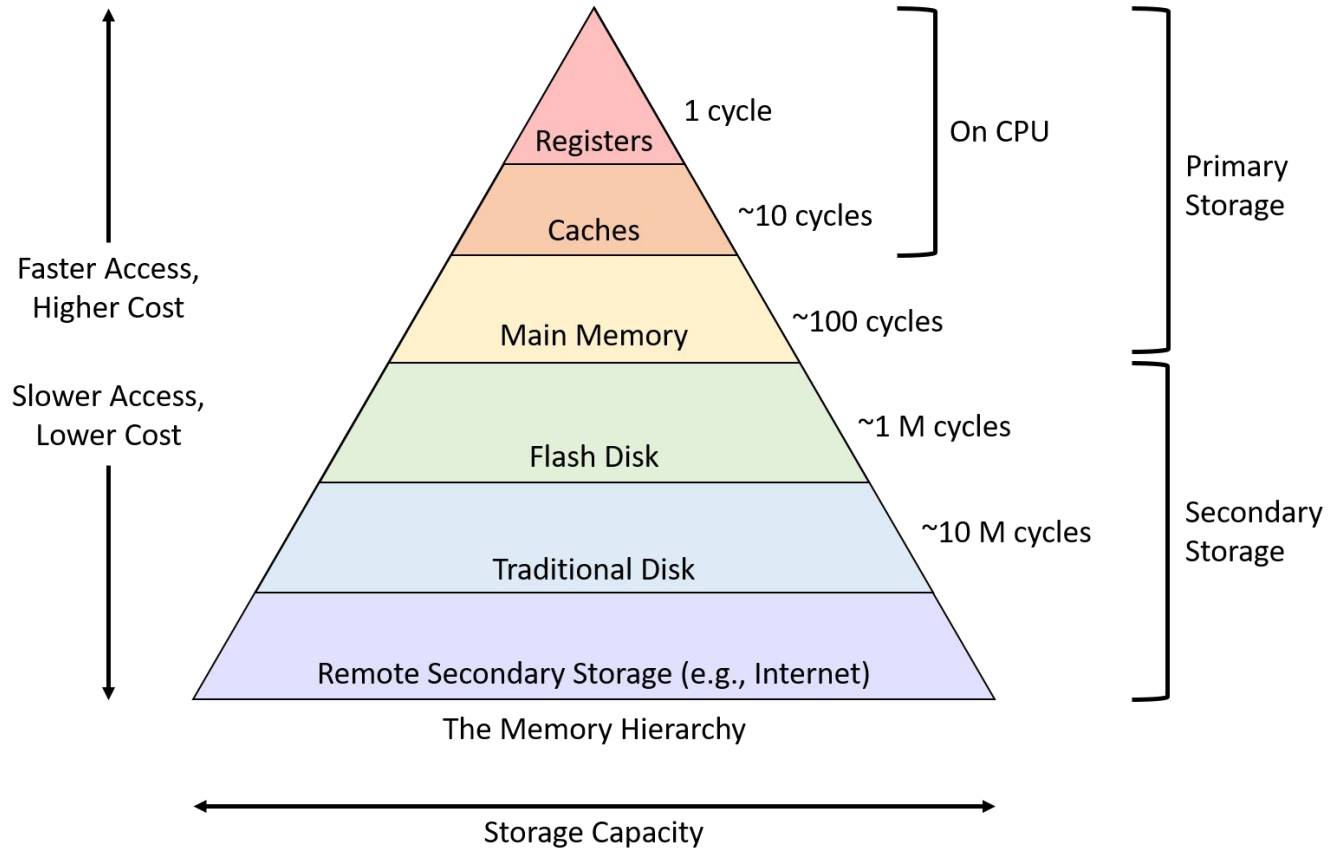
5. Segmentation

Segmented memory management further breaks down the paging technique, allowing for the segmenting of blocks into various sizes, but a maximum size limit exists for each segment.

3. MEMORY HIERARCHY

Memory hierarchy refers to the arrangement of different types of computer memory in a system, organized in ascending order based on their access speed, capacity, and cost. This concept is integral to computer architecture and aims to optimize the trade-offs between speed, size, and data storage and retrieval costs.

Memory hierarchy exploits the principle of locality, which states that programs tend to access a small portion of data frequently (temporal locality) and access data that is located close to recently accessed data (spatial locality). This principle guides data placement across different memory levels to improve overall system performance.



Aspects of Memory Hierarchy

- Speed
- Capacity
- Cost per bit
- Access Latency
- Persistence or Volatility

Why is the memory hierarchy important?

- Bridges the Performance Gap
Processors execute instructions much faster than main memory can supply data. Memory hierarchy reduces this mismatch by placing faster, smaller memory closer to the CPU.
- Optimizes System Performance
Frequently accessed data is stored in faster memory levels (registers, cache).
Less frequently accessed data is stored in larger, slower memory levels (RAM, storage).
- Reduces CPU Idle Time
Faster memory levels minimize the time the CPU waits for data, improving efficiency.

Memory Interfacing

When we are executing any instruction, we need the microprocessor to access the memory for reading instruction codes and the data stored in the memory. For this, both the memory and the microprocessor requires some signals to read from and write to registers.

Memory interfacing refers to connecting the memory chips to a microprocessor so that the CPU can read and write data properly.

How Does It Work?

- Connections: Physical and logical pathways (e.g., buses, interconnects) are established between memory and other components.
- Protocols: Communication protocols (e.g., DDR, PCIe) define how data is transferred.
- Control Signals: Signals like read/write and address lines coordinate data transfer.

Effective Memory Access Time (EMAT)

EMAT is the average time it takes for the computer system to access data from memory, considering cache hits, cache misses, and memory latency.

Components of EMAT

- Cache Access Time: Time to access data if it is found in the cache.
Ex: L1 cache access time is ~1-2 ns, while L2 cache is ~5-10 ns.
- Cache Hit Rate: Percentage of memory accesses where data is found in the cache. A higher hit rate reduces EMAT.
Ex: A 95% cache hit rate means 95% of accesses are served by the cache.
- Cache Miss Penalty: Additional time required to fetch data from main memory when a cache miss occurs.
Ex: If cache access time is 5 ns and main memory access time is 100 ns, the miss penalty is 95 ns.
- Main Memory Access Time: Time to access data directly from RAM.
Ex: DDR4 RAM has an access time of ~50-100 ns.

To calculate the effective memory access time, the cache hit rate and cache miss penalty are taken into account. The formula for EMAT is:

$$\text{EMAT} = (\text{Cache Hit Rate} \times \text{Cache Access Time}) + (\text{Cache Miss Rate} \times \text{Cache Miss Penalty})$$

Factors Influencing EMAT

- Cache Size: Larger caches improve hit rates but increase access time.
- Cache Hierarchy: Multi-level caches (L1, L2, L3) optimize EMAT by balancing speed and size.
- TLB Hit Ratio (Probability): The percentage of time a virtual address translation is found in the high-speed TLB cache. A higher hit ratio significantly reduces EMAT.
- Page Fault Rate: The probability that a required page is not in the physical RAM, necessitating a slow access to the hard disk (swap space).
- Main Memory Access Time ($\frac{1}{M}$)

Activity

1. Given the following parameters for a memory system, can you calculate the Effective Memory Access Time (EMAT) using the formula?

Cache Hit Rate = 0.92

Cache Access Time = 2 ns

Cache Miss Rate = 0.08

Cache Miss Penalty = 25 ns

2. A cache memory needs an access time of 5 ns and main memory 100 ns, what is the average access time of CPU? (assume hit ratio = 95%)

3. Assume that for a certain processor, a read request takes 70ns on a cache miss and 4ns on a cache hit. Suppose while running a program, it was observed that 90% of the processor's read requests result in a cache hit. What is the average read access time in nanoseconds?

Examples of EMAT Optimization

- Gaming Consoles
- Data Centers
- Mobile Devices
- Automotive ECUs (Electronic Control Units) – Modern vehicles use ADAS (Advanced Driver-Assistance Systems) with tightly coupled memory and dedicated hardware accelerators to reduce EMAT for real-time sensor data processing.
- High-Performance Computing (HPC)
- Artificial Intelligence / Machine Learning
- Server & Cloud Computing

4. CACHE MEMORY

Overview of Caching

- Caching is a technique used to improve data access and retrieval speeds by storing copies of frequently accessed or recently used data in a faster and closer location (cache).
- Reduces the need to access slower storage (e.g., main memory or secondary storage).
- Bridges the performance gap between fast processors and slower memory systems.
- Accessing data from a cache is faster than retrieving it from the original source due to the cache's proximity to the processor and higher access speeds.

Defining Cache Memory

- Cache memory is a small, fast type of memory located close to the processor.
- Stores frequently accessed instructions and data to provide faster access compared to main memory.
- Acts as a temporary storage buffer between the processor and main memory.
- Exploits the principle of locality of reference to optimize performance.

What is locality in the context of cache memory?

Locality of reference is a crucial concept to grasp when discussing cache memory's role in enhancing a computer's performance. Locality refers to the predictable patterns found in memory access, which cache memory exploits to optimize data retrieval processes.

- **Temporal Locality:** This occurs when certain data is accessed repeatedly over a short period. For instance, if a program frequently accesses a specific variable, temporal locality ensures that the necessary data remains readily accessible, minimizing the time needed to retrieve it.
- **Spatial Locality:** This involves accessing a sequence of data elements that are physically close to each other in memory. For example, when traversing an array or a list, the memory locations are often consecutively aligned, allowing the cache to load them efficiently in one go.

Levels of Cache

Modern systems use multiple levels of cache (L1, L2, L3) organized hierarchically

- L1 Cache: Smallest and fastest, integrated directly into the CPU.
- L2 Cache: Larger but slower than L1, often shared between CPU cores.
- L3 Cache: Largest but slower than L2, shared across all cores.

Ex:

ARM Cortex-A78 (Mobile Processor)

L1 Instruction; 32 KB, 1-2 cycles, Per core, Holds the next few instructions to execute

L1 Data; 32 KB, 1-2 cycles, Per core, Holds frequently used data variables

L2 Cache; 256 KB, 8-10 cycles, Per core, Catches misses from L1

L3 Cache; 2-4 MB, 20-30 cycles, Shared across all cores, Catches misses from L2; Main Memory (LPDDR5); 8-16 GB, 200-300 cycles, Separate chip, Final level before disk

Beyond the typical levels of cache memory, there are specialized caches that serve distinct functions within a system. For example, translation lookaside buffers (TLBs) are specialized memory caches that map virtual addresses to physical addresses, facilitating quicker data retrieval.

Moreover, some caches aren't purely memory caches. Disk caches, for instance, use DRAM or flash memory to store frequently accessed disk data, speeding up retrieval times similar to CPU instruction caches. This highlights the adaptability of caching technology across various types of storage.

Specialized caches are also integral to applications like web browsers, databases, and network protocols. These caches can be distributed across multiple networked hosts, enhancing scalability and performance for the applications that rely on them. By optimizing data access and storage, these specialized caches ensure smoother and more efficient operation across diverse computing environments.

How Caching Works

1. Data Access Process

When the processor needs data, it checks the cache hierarchy starting from L1.
Compares the memory address of the requested data with addresses stored in the cache.

2. Cache Hit

If the data is found in the cache, it is a cache hit.
Data is retrieved from the cache, reducing access time and improving performance.

3. Cache Miss

If the data is not found in the cache, it is a cache miss.
Data is fetched from main memory, introducing additional latency (cache miss penalty).

4. Storing Frequently Accessed Data

The cache dynamically identifies and stores frequently accessed data based on access patterns.
Uses caching strategies and replacement policies to manage cache contents.

Data writing policies in cache memory

Write-Through

- Simultaneous Update: When employing the write-through policy, data is simultaneously updated in both the cache and the main memory.
- Immediate Consistency: This ensures data consistency, but often results in a latency increase due to the double-write requirement.

Write-Back

- Cached Priority: With write-back, data is initially written only to the cache. It is then later pushed to the main memory, if needed.
- Efficiency: This approach can enhance performance by reducing immediate writing demands, but it risks occasional inconsistencies between cache and main memory.

Caching in Real-World Systems

- Web Browsers
Store frequently accessed web pages and resources (e.g., images, scripts) in a browser cache to reduce load times.
- Content Delivery Networks (CDNs)
Use edge caching to store copies of content closer to users, reducing latency and improving load times.
Example: Cloudflare and Akamai CDNs.
- Databases
Use query caching to store results of frequently executed queries, improving response times.
Example: MySQL and Redis caching mechanisms.
- Gaming Consoles
Use custom SSDs and high-speed memory to cache game assets, enabling faster loading and rendering.
Example: PlayStation 5 and Xbox Series X.

Emerging Cache Types

- Non-Volatile Cache (NVM); Replaces power-hungry SRAM with high-density, non-volatile memory like STT-MRAM or SOT-MRAM.
- Approximate Cache; A cache that can be intentionally "imprecise" with less important data (e.g., image pixels) for massive energy savings
- Hybrid Cache (SRAM + NVM); Combines fast, durable SRAM with dense, energy-efficient NVM to get the "best of both worlds"
- CXL & Tiered Caching; Uses new high-speed interconnect (CXL) to efficiently use far memory (like SSDs) as a large, slow cache layer
- TDV Cache (Fusion Cache); A smart design using direct-mapped (fast) + victim cache (high hit rate) + tag cache (efficient)

5. VIRTUAL MEMORY

It is a Memory management technique that combines RAM and secondary storage to create an expanded memory address space.

The key mechanism of Virtual Memory is using demand paging and address translation to map virtual addresses to physical locations.

Two Main Components for Implementation

Memory Management Unit (MMU): A hardware unit built directly into the CPU that performs address translation on the fly for every memory access.

Operating System (OS): Manages page tables, handles page faults, and decides which pages stay in RAM versus which are moved to disk.

How a Virtual Address is Translated

Say that,

Page size = 4KB (2^{12} bytes → 12-bit offset)

Logical address = 0x3A7F (16-bit address)

1. First we split into Page Number & Offset

Total bits = 16

Offset bits = 12

Page number bits = $16 - 12 = 4$ bits

Logical address: 0x3A7F → Binary: 0011 1010 0111 1111

Page Number (4 bits): 0011 (3 in decimal)

Offset (12 bits): 1010 0111 1111 (0xA7F in hex)

2. Then, we Look Up Page Table

Assume the page table maps:

Page #	Frame #
--------	---------

0	5
---	---

1	2
---	---

2	7
---	---

3	1
---	---

Page 3 → Frame 1

3. Finally, we construct the Physical Address

Frame Number (1) + Offset (0xA7F)

Physical address = $(1 \ll 12) + 0xA7F = 0x1A7F$

Further reading:

<https://www.cs.cornell.edu/courses/cs4410/2015su/lectures/lec14-pagetables.html>

Each Page Table Entry (PTE) contains:

- A valid bit (1 = page is in RAM, 0 = page is on disk)
- A Physical Page Number (PPN) (if valid = 1)
- Protection bits (read, write, execute permissions)
- Dirty bit (has the page been modified while in RAM?)
- Referenced bit (has the page been accessed recently?)

Benefits of Virtual Memory

1. Large Program Support:

Virtual memory enables the execution of larger programs that may not fit entirely in the limited physical memory. It allows the system to store and manage data on the disk, effectively extending the available memory space.

Example: Running 16GB applications on 8GB RAM systems

Modern development: Memory compression before swapping

2. Advanced Multitasking:

Virtual memory facilitates efficient multitasking by allowing multiple programs to run simultaneously. Each program can have its own virtual memory space, isolating it from other programs and preventing interference.

Example: Chrome using separate address spaces per tab

New feature: Memory deduplication (KSM in Linux)

3. Enhanced Memory Protection:

Virtual memory provides memory protection by assigning memory spaces to different processes. It ensures that each process can only access its allocated memory, preventing unauthorized access and enhancing system stability.

Ex: Protection against Meltdown/Spectre vulnerabilities

4. Efficiency Optimizations:

Virtual memory optimizes memory utilization by keeping only the most frequently used data in physical memory.

Less frequently used data is swapped out to disk, freeing up space for other processes to use.

Ex: Android's LMKD (Low Memory Killer Daemon) improvements

5. Programming Advantages:

Simplifies programming by providing a consistent and uniform memory model for applications.

Ex: `mmap()` for fast database access

Page Faults and Swapping

Page Faults

Page fault dominates more like an error. It mainly occurs when any program tries to access the data or the code that is in the address space of the program, but that data is not currently located in the RAM of the system.

Types:

- Minor Fault: Page is in RAM but not mapped (fast resolution).
- Major Fault: Page must be loaded from disk (slow, $\sim 10\text{-}50\mu\text{s}$ on NVMe).
- Invalid Fault: Illegal access.

Handling the Page Fault

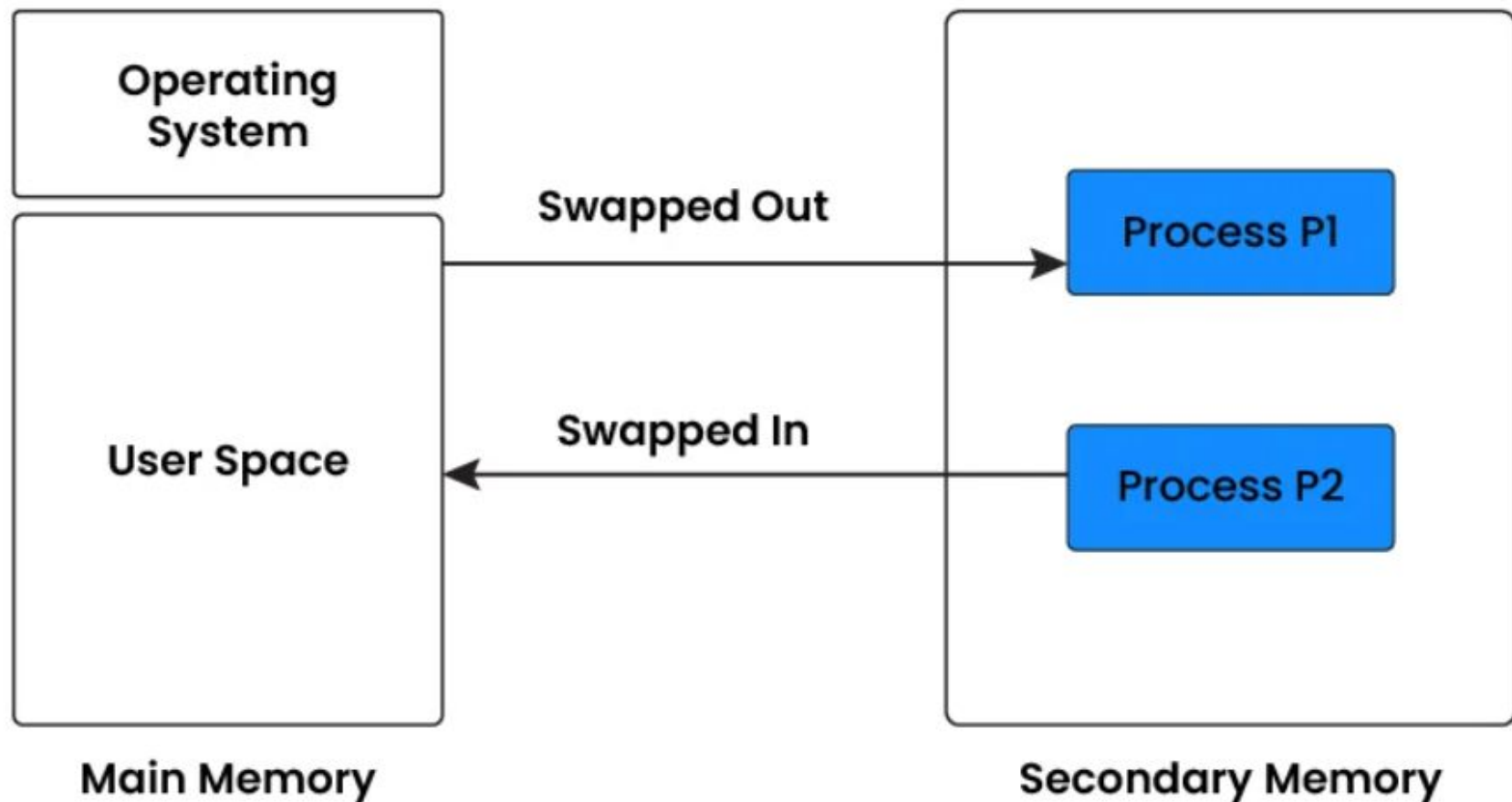
1. First of all, internal table(that is usually the process control block) for this process determine whether the reference was valid or invalid memory access.
2. If the reference is invalid, then we will terminate the process. If the reference is valid, but we have not bought in that page so now we just page it in.
3. Then we locate the free frame list in order to find the free frame.
4. Now a disk operation is scheduled in order to read the desired page into the newly allocated frame.
5. When the disk is completely read, then the internal table is modified that is kept with the process, and the page table that mainly indicates the page is now in memory.
6. Now we will restart the instruction that was interrupted due to the trap. Now the process can access the page as though it had always been in memory.

Swapping

Swapping occurs when the system does not have enough space in the main memory or the RAM, and it needs to temporarily move the data or program that is not currently being utilized to a designated place on the hard disk called swap space. The process of interchanging data back between the RAM and swap space is called swapping, and it allows a user to run multiple programs at once, even if they do not have enough RAM.

Assume the processes P1 is in the main memory, but P2 needs to be executed. The operating system swaps out P1 to the secondary memory (swap space) to free physical memory for process P2. Then the process P2 is loaded into main memory, and the execution begins. When process P1 is again needed, the operating system will swap P1 back from swap space.

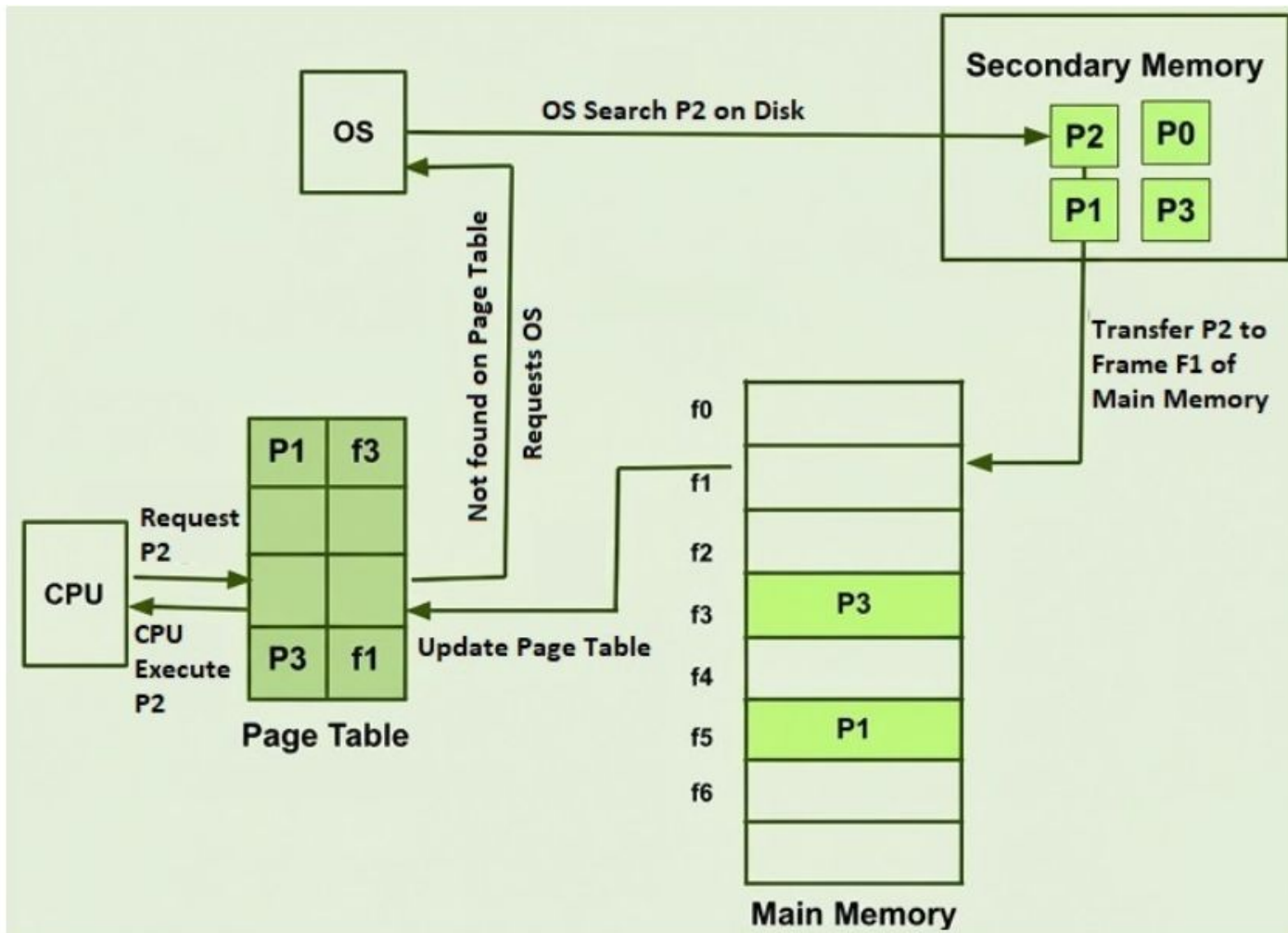
Swapping of Processes



Demand Paging

Demand paging is an optimization done on top of the paging memory management method. In demand paging, the pages of a process are loaded into memory only when they are needed (i.e., on demand) instead of loading all pages at once when the process starts.

Suppose that a process is trying to access a page that is not currently in physical memory (RAM). It will lead to a page fault. So, the operating system loads the required pages from the virtual memory (disk) into physical memory. This happens on demand of process, hence the name demand paging. If the frames of the physical memory is full during the loading of new pages, then the OS uses page replacement algorithms to decide which pages are needed to be removed to make space for the new page.



How demand paging work

CPU request Page 2 – First, the CPU requests Page 2. The page table is used to check if Page 2 is in main memory. In this case, it is not. So a page fault occurs.

Page Fault Handling – The operating system is notified of the page fault. It locates Page 2 on the disk and finds a free frame in main memory.

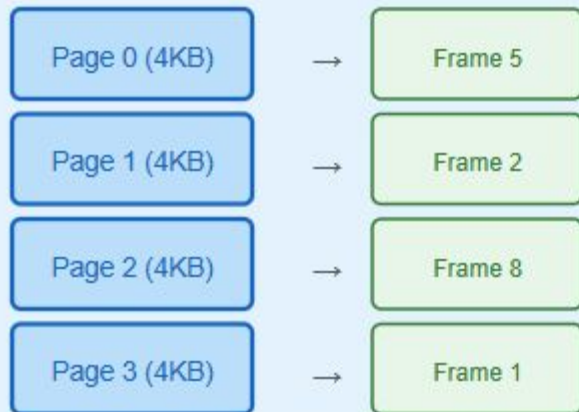
Loading the Page – The OS loads Page 2 from the disk into the free frame in main memory. Then the page table is updated with the new frame number for Page 2.

Resuming the Process – Finally, the CPU is notified that Page 2 is now in memory, and the process can continue executing.

Paging vs. Segmentation

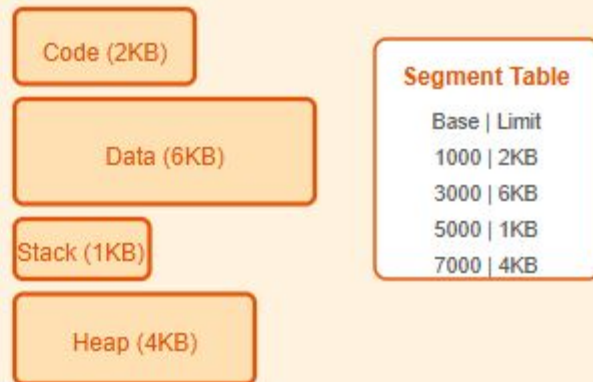
Feature	Paging	Segmentation
Block Size	Fixed-size pages	Variable-size segments
Managed By	Operating system	Compiler (calculates segment sizes and addresses)
Size Determined By	Available memory (hardware)	Logical structure of the program (user/compiler)
Speed	Faster memory access	Slower than paging
Fragmentation	Internal fragmentation (underutilized pages)	External fragmentation (unused memory gaps)
Logical Address	Page number + page offset	Segment number + segment offset
Lookup Table	Page table (page → frame mapping)	Segment table (base address + length)

Paging (Fixed Size)



All blocks same size

Segmentation (Variable Size)



Segment Table

Base	Limit
1000	2KB
3000	6KB
5000	1KB
7000	4KB

Blocks vary in size

Virtual Memory Performance Considerations

Page Table Optimizations

- Multi-Level Page Tables (Hierarchical Paging): Instead of one massive contiguous table, this breaks the table into smaller, tree-structured pages. This saves memory because lower-level tables are only allocated when needed.
- Inverted Page Tables: Instead of one table per process, a single table is maintained for the entire system, containing entries for only physical frames. This dramatically reduces memory footprint, especially with many processes.
- Translation Lookaside Buffer (TLB) Caching: A specialized, high-speed hardware cache that stores recent virtual-to-physical address translations. It dramatically reduces the need to access main memory to read the page table.

Page Replacement Algorithms:

- First In First Out (FIFO): Replaces the oldest page that was loaded into memory first. It is simple to implement using a queue but can result in high page fault rates and suffers from Belady's Anomaly.
- Optimal Page Replacement (OPR): Selects the page that will not be used for the longest period in the future. While it guarantees the lowest possible page fault rate, it is largely theoretical because it requires future knowledge of page requests.
- Least Recently Used (LRU): Replaces the page that hasn't been accessed for the longest time. It operates on the assumption that pages used recently will likely be used again soon, offering better performance than FIFO.
- Most Recently Used (MRU): Replaces the page that was accessed most recently.
- Least Frequently Used (LFU): Replaces the page with the lowest access count.

Disk I/O Overhead Mitigation

Accessing data from secondary storage is slower than accessing data from RAM, so excessive page faults can lead to performance degradation.

NVMe & Optane – Faster than HDDs ($\sim 10\mu\text{s}$ vs. 10ms).

Zswap/ZRAM – Compress pages in RAM before swapping.

Example: MacBooks use NVMe SSDs + compression for smooth swapping.

Security Enhancements

Memory Tagging (ARM MTE) – Detects buffer overflows.

KASLR (Kernel ASLR) – Randomizes kernel memory layout.

THANK YOU!

23.06.2026